

CM07 - Arrays and Strings

Arrays, arrays and pointers, strings, 2D arrays

Louis Ledoux

2026-07-05

Table of contents

1	Array Type	2
1.1	Main Path	2
1.2	Worked Example	2
1.3	Anecdote Or Motivation	3
1.4	Check Question	3
1.5	Backup Index	4
1.6	Backup: Array type	6
1.7	Backup: Array type: Definition	7
1.8	Backup: Arrays and Pointers	8
1.9	Backup: Arrays and Pointers	9
1.10	Backup: Arrays and Pointers	11
1.11	Backup: An array of pointers	12
2	Strings	12
2.1	Main Path	12
2.2	Worked Example	13
2.3	Anecdote Or Motivation	13
2.4	Check Question	14
2.5	Backup Index	14
2.6	Backup: An array of characters: String	17
2.7	Backup: String initialisation	18
2.8	Backup: String: Remarks	19
2.9	Backup: Arrays and Pointers: Attention !	20
2.10	Backup: Arrays and Pointers: Attention !	20
2.11	Backup: Let's...	21
3	Two-dimensional Arrays	21
3.1	Main Path	22
3.2	Worked Example	22
3.3	Anecdote Or Motivation	23
3.4	Check Question	23
3.5	Backup Index	24
3.6	Backup: 2D Array: Array of Arrays	26
3.7	Backup: Array of Arrays: Initialization	27
3.8	Backup: Array of Arrays	28
3.9	Backup: Array of Arrays	29
3.10	Backup: Array of Arrays: Remarks!	30
3.11	Backup: Array of Arrays: Pointer arithmetic	31
3.12	Backup: Array of Arrays: Pointer arithmetic	32
3.13	Backup: Array of Arrays: Pointer arithmetic	34
3.14	Backup: Pointer to an array	35

1 Array Type

Live pacing: about 8 min

Question. When does an array behave like a pointer, and when does it not?

Core claim. Arrays are contiguous objects; many expressions expose the address of their first element.

Target capability. Students can draw array storage, pointer arithmetic, strings, and 2D indexing.

1.1 Main Path

- Start from the question: When does an array behave like a pointer, and when does it not?
- Build the model: Arrays are contiguous objects; many expressions expose the address of their first element.
- End with a checkable action: Students can draw array storage, pointer arithmetic, strings, and 2D indexing.
- Keep only this slide on the horizontal path during the live lecture.

Array type

- Syntax:
- `type identifier[SIZE];`
- Function:
- The identifier is an array with a size equal to SIZE
- Each array element is a object of type type
- The array is statically allocated. During definition
- The SIZE must be is a constant
- The memory to store SIZE elements of type type is reserved
- The name of the array (identifier) is a constant pointer that points to the address of the first element of the array
- The index of the array is an integer
- Values from 0 to SIZE-1
- It can be initialized during its definition (SIZE=elements) `int array[]={1,3,-2,19,-79};`

#222

1.2 Worked Example

- Use this as the live trace: Use `arr`, `&arr[0]`, `arr + 1`, then repeat with a string ending in `\0`.
- Representative source slide: 222
- Ask students to predict the next state before revealing the explanation.
- Then connect the result back to the core claim.

Array type

- Syntax:
- `type identifier[SIZE];`
- Function:
- The identifier is an array with a size equal to SIZE
- Each array element is a object of type type
- The array is statically allocated. During definition
- The SIZE must be a constant
- The memory to store SIZE elements of type type is reserved
- The name of the array (identifier) is a constant pointer that points to the address of the first element of the array
- The index of the array is an integer
- Values from 0 to SIZE-1
- It can be initialized during its definition (SIZE=elements) `int array[]={1,3,-2,19,-79};`

#222

1.3 Anecdote Or Motivation

A string is not magic in C; it is an array convention plus library functions that respect it.

- Use this only if students need a reason to care.
- If the room is already following, skip down to the check question.

1.4 Check Question

- Ask for a prediction, not a definition.
- Require a short justification: command trace, memory drawing, type rule, or file state.
- If more than a third of the room hesitates, go down to the source backup.

An array of pointers

- The type of the elements of an array can be also pointers, that is of type `type *`
- Example:
- `int **arr[10];`
- `int **ptr = arr;`
- An element in the array `arr` (e.g. `arr[0]`) is of type `int*`

#227

Original slides 222-227

#222

#223

4

Arrays and Pointers

- The notion of array is very close to the notion of pointer
- `arr[i] == *(arr+i)`, so they indicate the same element!
- `arr+i = arr + sizeof(type)*i`

- Memory

- `int arr[10];`

- arr	- arr[0]:	- 0x1000	- 0
	- arr[1]:	- 0x1004	- 1
	- arr[2]:	- 0x1008	- 2
- arr+3	- arr[3]:	- 0x100C	- 3
		- ...	- 4
			- 5
	- *(arr+3)		- 6
			- 7
			- 8
	- arr[9]:	- 0x1028	- 9

arr is a constant pointer

arr++ is not allowed

#224

slide 224

Arrays and Pointers

- We can use a pointer to point to the array

- `int arr[10];`
- `int *ptr = arr;`

- Memory

- ptr			
- arr	- arr[0]:	- 0x1000	- 0
- ptr++	- arr[1]:	- 0x1004	- 1
- ptr++	- arr[2]:	- 0x1008	- 2
- ptr++	- arr[3]:	- 0x100C	- 3
		- ...	- 4
			- 5
			- 6
			- 7
			- 8
	- arr[9]:	- 0x1028	- 9

#225

slide 225

Arrays and Pointers

- What is the difference ?
- When we define an array:
 - We reserve one memory block equal to the size required to store the values of all the array elements
 - The array name is a constant pointer, its value cannot be modified
- When we define a pointer:
 - we do NOT reserve the memory where we will store the values of the array elements
 - We reserve ONLY the memory, where we will store the pointer
 - The pointer is a variable, its value can be modified

#226

slide 226

An array of pointers

- The type of the elements of an array can be also pointers, that is of type `type *`
- Example:
 - `int *arr[10];`
 - `int **ptr = arr;`
- An element in the array `arr` (e.g. `arr[0]`) is of type `int *`

#227

slide 227

anchors: Array type; Array type: Definition; Arrays and Pointers; An array of pointers

1.6 Backup: Array type

Original slide 222

- Syntax:
 - `type identifier[SIZE];`
- Function:
 - The identifier is an array with a size equal to `SIZE`
 - Each array element is a object of type `type`

- The array is statically allocated. During definition
- The SIZE must be is a constant
- The memory to store SIZE elements of type type is reserved
- The name of the array (identifier) is a constant pointer that points to the address of the first element of the array
- The index of the array is an integer
- Values from 0 to SIZE-1
- It can be initialized during its definition (SIZE=elements) `int array[]={1,3,-2,19,-79};`

Array type

- Syntax:
- `type identifier[SIZE];`
- Function:
- The identifier is an array with a size equal to SIZE
- Each array element is a object of type type
- The array is statically allocated. During definition
- The SIZE must be is a constant
- The memory to store SIZE elements of type type is reserved
- The name of the array (identifier) is a constant pointer that points to the address of the first element of the array
- The index of the array is an integer
- Values from 0 to SIZE-1
- It can be initialized during its definition (SIZE=elements) `int array[]={1,3,-2,19,-79};`

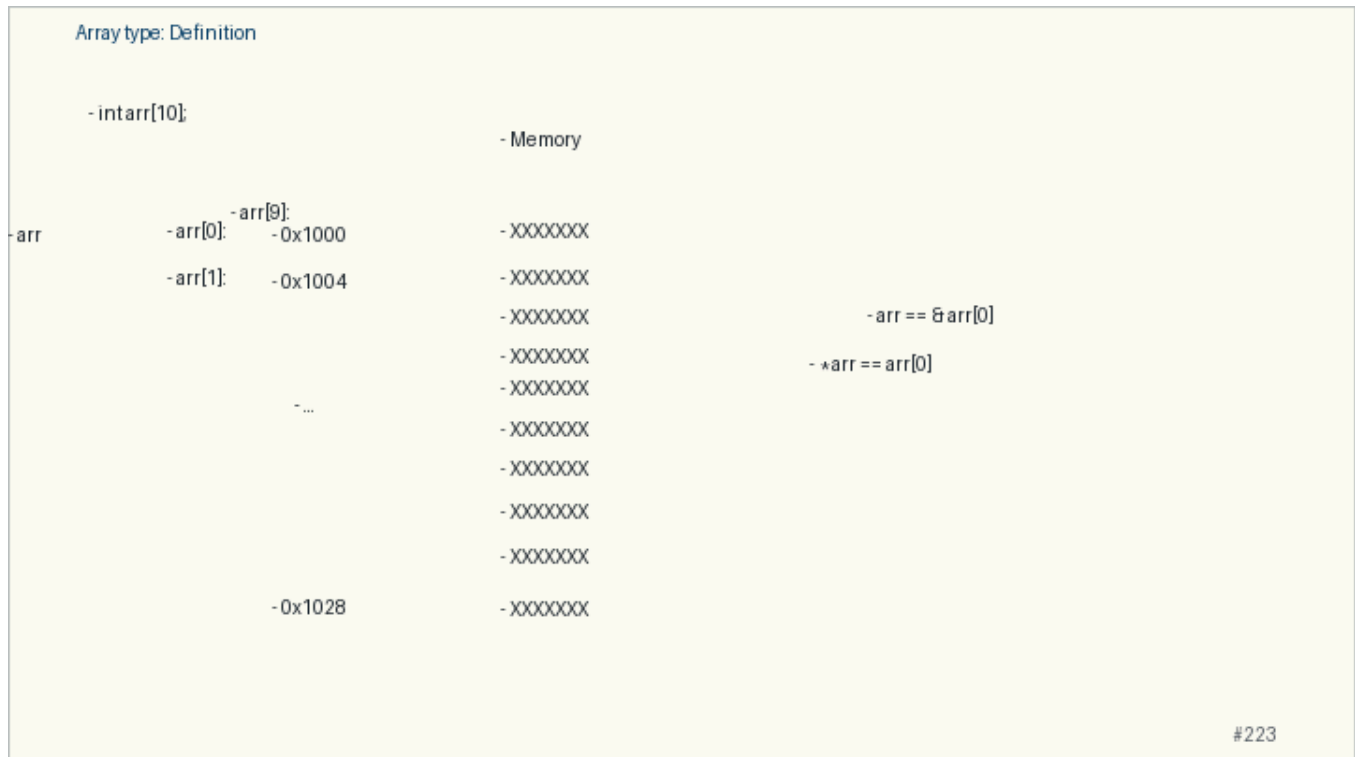
#222

1.7 Backup: Array type: Definition

Original slide 223

- `int arr[10];`
- Memory
- `arr[9]:`
- `arr[0]:`
- XXXXXXXX
- 0x1000
- `arr`
- `arr[1]:`
- XXXXXXXX
- 0x1004
- `arr == & arr[0]`
- XXXXXXXX
- XXXXXXXX
- `*arr == arr[0]`
- XXXXXXXX

- ...
- XXXXXXXX
- XXXXXXXX
- XXXXXXXX
- XXXXXXXX
- 0x1028
- XXXXXXXX



1.8 Backup: Arrays and Pointers

Original slide 224

- The notion of array is very close to the notion of pointer
- `arr[i] == *(arr+i)` , so they indicate the same element !
- `arr+i = arr + sizeof(type)*i`
- Memory
- `int arr[10];`
- `arr`
- `arr[0]:`
- 0
- 0x1000
- `arr[1]:`
- 1
- 0x1004
- `arr[2]:`

- 2
- 0x1008
- arr[3]:
- 0x100C
- 3
- arr+3
- 4
- ...
- 5
- arr is a constant pointer
- ->
- arr++ is not allowed
- 6
- *(arr+3)
- 7
- 8
- 9
- 0x1028
- arr[9]:

Arrays and Pointers

- The notion of array is very close to the notion of pointer
- arr[i] == *(arr+i), so they indicate the same element!
- arr+i = arr + sizeof(type) * i

				- Memory
- int arr[10];				
	- arr	- arr[0]:	- 0x1000	- 0
		- arr[1]:	- 0x1004	- 1
		- arr[2]:	- 0x1008	- 2
	- arr+3	- arr[3]:	- 0x100C	- 3
			- ...	- 4
				- 5
arr is a constant pointer		- *(arr+3)		- 6
arr++ is not allowed				- 7
				- 8
		- arr[9]:	- 0x1028	- 9

#224

1.9 Backup: Arrays and Pointers

Original slide 225

- We can use a pointer to point to the array

- `int arr[10];`
- `int *ptr = arr;`
- Memory
- `ptr`
- `arr`
- `arr[0]:`
- 0
- 0x1000
- `arr[1]:`
- 1
- `ptr++`
- 0x1004
- 2
- `arr[2]:`
- 0x1008
- `ptr++`
- 3
- `arr[3]:`
- 0x100C
- `ptr++`
- 4
- ...
- 5
- 6
- 7
- 8
- 9
- 0x1028
- `arr[9]:`

Arrays and Pointers

- We can use a pointer to point to the array

```
- int arr[10];  
- int *ptr = arr;
```

- ptr		- Memory	
- arr	- arr[0]:	- 0x1000	- 0
- ptr++	- arr[1]:	- 0x1004	- 1
- ptr++	- arr[2]:	- 0x1008	- 2
- ptr++	- arr[3]:	- 0x100C	- 3
		- ...	- 4
			- 5
			- 6
			- 7
			- 8
	- arr[9]:	- 0x1028	- 9

#225

1.10 Backup: Arrays and Pointers

Original slide 226

- What is the difference ?
- When we define an array:
 - We reserve one memory block equal to the size required to store the values of all the array elements
 - The array name is a constant pointer, its value cannot be modified
- When we define a pointer:
 - we do NOT reserve the memory where we will store the values of the array elements,
 - We reserve ONLY the memory, where we will store the pointer
- The pointer is a variable, its value can be modified

Arrays and Pointers

- What is the difference ?
- When we define an array:
 - We reserve one memory block equal to the size required to store the values of all the array elements
 - The array name is a constant pointer, its value cannot be modified
- When we define a pointer:
 - we do NOT reserve the memory where we will store the values of the array elements,
 - We reserve ONLY the memory, where we will store the pointer
- The pointer is a variable, its value can be modified

#226

1.11 Backup: An array of pointers

Original slide 227

- The type of the elements of an array can be also pointers, that is of type `type*`
- Example:
 - `int *arr[10];`
 - `int **ptr = arr;`
- An element in the array `arr` (e.g. `arr[0]`) is of type `int*`
- The constant pointer `arr` is the address of the first element of the array
- Hence, the `ptr` is a pointer to a pointer of integers
- `ptr+i = ptr + sizeof(int*) * i`

An array of pointers

- The type of the elements of an array can be also pointers, that is of type `type *`
- Example:
 - `int *arr[10];`
 - `int **ptr = arr;`
- An element in the array `arr` (e.g. `arr[0]`) is of type `int *`

#227

2 Strings

Live pacing: about 8 min

Question. When does an array behave like a pointer, and when does it not?

Core claim. Arrays are contiguous objects; many expressions expose the address of their first element.

Target capability. Students can draw array storage, pointer arithmetic, strings, and 2D indexing.

2.1 Main Path

- Start from the question: When does an array behave like a pointer, and when does it not?
- Build the model: Arrays are contiguous objects; many expressions expose the address of their first element.
- End with a checkable action: Students can draw array storage, pointer arithmetic, strings, and 2D indexing.
- Keep only this slide on the horizontal path during the live lecture.

An array of characters: String

There is no string type in C
A string is an array of characters finishing with '\0'
char string[SIZE]
The SIZE must be large enough to hold this additional byte!
Compiler puts the '\0' character at the first empty element of the array
No automatic array bound checking!
If number of characters and '\0' > array maximum size
UNDEFINED BEHAVIOUR
Responsibility of the programmer:
content of the string <= size reserved in the array

- This is the source of
- A LOT of ERRORS!

- SECURITY ISSUES

#228

2.2 Worked Example

- Use this as the live trace: Use `arr`, `&arr[0]`, `arr + 1`, then repeat with a string ending in `\0`.
- Representative source slide: 231
- Ask students to predict the next state before revealing the explanation.
- Then connect the result back to the core claim.

Arrays and Pointers: Attention !

- Example:
- `int arr[10];`
- `int *ptr1, *ptr2;`
- `ptr1 = arr + 3; // ptr1 == &arr[3]`
- `ptr2 = ptr1 - 2; // ptr2 == &arr[1];`

#231

2.3 Anecdote Or Motivation

A string is not magic in C; it is an array convention plus library functions that respect it.

- Use this only if students need a reason to care.
- If the room is already following, skip down to the check question.

2.4 Check Question

- Ask for a prediction, not a definition.
- Require a short justification: command trace, memory drawing, type rule, or file state.
- If more than a third of the room hesitates, go down to the source backup.

Let's...

- Kahoot time !!
- Q7 - Q11

#233

2.5 Backup Index

Original slides 228-233

An array of characters: String

There is no string type in C
A string is an array of characters finishing with '\0'
char string[SIZE]
The SIZE must be large enough to hold this additional byte!
Compiler puts the '\0' character at the first empty element of the array
No automatic array bound checking!
If number of characters and '\0' > array maximum size
UNDEFINED BEHAVIOUR
Responsibility of the programmer:
content of the string <= size reserved in the array

- This is the source of
- A LOT of ERRORS!

- SECURITY ISSUES

#228

slide 228

String initialisation

```
char c[] = "abcde";  
char c[6] = "abcde";  
char c[] = {'a','b','c','d','e','\0'};
```

- Memory

```
-c      -c[0]:  -0x100C      -a      -b      -c      -d
      -e      -\0
```

#229

slide 229

String: Remarks

```
Static declaration
char str1[] = "hello";
char str2[] = "world";
We CANNOT assign a string to another (str2 = str1;). They are constant pointers!
We have to copy one to the other!
Once the string is initialized, it CANNOT be assigned to another set of characters
str1 = "bye";
```

#230

slide 230

Arrays and Pointers: Attention !

```
- Example:  
- int arr[10];  
- int *ptr1, *ptr2;  
- ptr1 = arr + 3; //ptr1 == &arr[3]  
- ptr2 = ptr1 - 2; //ptr2 == &arr[1];
```

#231

slide 231

Arrays and Pointers: Attention !

```
- Example:  
- int arr[10];  
- int *ptr1, *ptr2;  
- ptr1 = arr + 30; //ptr1 == &arr[30]  
- ptr2 = ptr1 - 2; //ptr2 == &arr[28]
```

#232

slide 232

Let's..

- Kahoot time !!
- Q7 - Q11

#233

slide 233

Anchors: An array of characters: String; String initialisation; String: Remarks; Arrays and Pointers: Attention !;
Let's...

2.6 Backup: An array of characters: String

Original slide 228

```
There is no string type in C
A string is an array of characters finishing with '\0'
char string[SIZE]
The SIZE must be large enough to hold this additional byte!
Compiler puts the '\0' character at the first empty element of the array
No automatic array bound checking!
If number of characters and '\0' > array maximum size
UNDEFINED BEHAVIOUR
Responsibility of the programmer:
content of the string <= size reserved in the array
```

- This is the source of
- A LOT of ERRORS !
- SECURITY ISSUES

An array of characters: String

There is no string type in C
A string is an array of characters finishing with '\0'
char string[SIZE]
The SIZE must be large enough to hold this additional byte!
Compiler puts the '\0' character at the first empty element of the array
No automatic array bound checking!
If number of characters and '\0' > array maximum size
UNDEFINED BEHAVIOUR
Responsibility of the programmer:
content of the string <= size reserved in the array

- This is the source of
- A LOT of ERRORS!

- SECURITY ISSUES

#228

2.7 Backup: String initialisation

Original slide 229

```
char c[] = "abcde";  
char c[6] = "abcde";  
char c[] = {'a', 'b', 'c', 'd', 'e', '\0'};  
char c[6] = {'a', 'b', 'c', 'd', 'e', '\0'};
```

- Memory

- a
- b
- c
- d
- c[0]:
- 0x100C
- c
- e
- \0

```
char c[] = "abcde";
char c[6] = "abcde";
char c[] = {'a','b','c','d','e','\0'};
```

```
-c      -c[0]:  -0x100C      -a      -b      -c      -d
                        -e      -\0
```

2.8 Backup: String: Remarks

Static declaration

```
char str2[]="world";
```

We have to copy one to the other!

Once the string is initialized, it CANNOT be assigned to another set of characters

```
str1 = "bye";
```

```
Static declaration
char str1[] = "hello",
char str2[] = "world";
```

Once the string is initialized, it **CANNOT** be assigned to another set of characters
 str1 = "bye";

19

2.9 Backup: Arrays and Pointers: Attention !

Original slide 231

- Example:
- `int arr[10] ;`
- `int *ptr1, *ptr2 ;`
- `ptr1 = arr + 3 ; //ptr1 == & arr[3]`
- `ptr2 = ptr1 - 2 ; //ptr2 == & arr[1] ;`
- Pointer ptr1 points the fourth element of the array
- Pointer ptr2 points two elements back from ptr1.
- Still, within the memory block of the array arr.

Arrays and Pointers: Attention !

```
- Example:  
- int arr[10];  
- int *ptr1, *ptr2;  
- ptr1 = arr + 3; //ptr1 == &arr[3]  
- ptr2 = ptr1 - 2; //ptr2 == &arr[1];
```

#231

2.10 Backup: Arrays and Pointers: Attention !

Original slide 232

- Example:
- `int arr[10] ;`
- `int *ptr1, *ptr2 ;`
- `ptr1 = arr + 30 ; //ptr1 == & arr[30]`
- `ptr2 = ptr1 - 2 ; //ptr2 == & arr[28]`
- Now, pointer ptr1 points the outside the memory block of the array !
- There is no compiler error, the compiler still computes the address!
- There is no core dump, if we are accessing the address space dedicated to the program!
- The data that we will read through *ptr1 are not valid!

Arrays and Pointers: Attention !

```
- Example:  
- int arr[10];  
- int *ptr1, *ptr2;  
- ptr1 = arr + 30; //ptr1 == &arr[30]  
- ptr2 = ptr1 - 2; //ptr2 == &arr[28]
```

#232

2.11 Backup: Let's...

Original slide 233

- Kahoot time !!
- Q7 - Q11

Let's..

```
- Kahoot time !!  
- Q7 - Q11
```

#233

3 Two-dimensional Arrays

Live pacing: about 8 min

Question. When does an array behave like a pointer, and when does it not?

Core claim. Arrays are contiguous objects; many expressions expose the address of their first element.

Target capability. Students can draw array storage, pointer arithmetic, strings, and 2D indexing.

3.1 Main Path

- Start from the question: When does an array behave like a pointer, and when does it not?
- Build the model: Arrays are contiguous objects; many expressions expose the address of their first element.
- End with a checkable action: Students can draw array storage, pointer arithmetic, strings, and 2D indexing.
- Keep only this slide on the horizontal path during the live lecture.

2D Array: Array of Arrays

- Syntax:
- `type identifier[ROWS][COLS];`
- A 2D array named `identifier` is defined as
- a number of ROWS
- Each row is an 1D array of COLS elements of type `type`
- When initializing during definition:
- It is always required to give the COLS dimension
- The number of ROWS is NOT always required
- it can be calculated based on the number of columns and the initialization.

#234

3.2 Worked Example

- Use this as the live trace: Use `arr`, `&arr[0]`, `arr + 1`, then repeat with a string ending in `\0`.
- Representative source slide: 234
- Ask students to predict the next state before revealing the explanation.
- Then connect the result back to the core claim.

2D Array: Array of Arrays

- Syntax:
- type identifier[ROWS][COLS];
- A 2D array named identifier is defined as
- a number of ROWS
- Each row is an 1D array of COLS elements of type type
- When initializing during definition:
- It is always required to give the COLS dimension
- The number of ROWS is NOT always required
- it can be calculated based on the number of columns and the initialization.

#234

3.3 Anecdote Or Motivation

A string is not magic in C; it is an array convention plus library functions that respect it.

- Use this only if students need a reason to care.
- If the room is already following, skip down to the check question.

3.4 Check Question

- Ask for a prediction, not a definition.
- Require a short justification: command trace, memory drawing, type rule, or file state.
- If more than a third of the room hesitates, go down to the source backup.

Let's..

- Kahoot time!
- Q12 - Q19

#243

3.5 Backup Index

Original slides 234-243

2D Array: Array of Arrays

- Syntax:
- `type identifier[ROWS][COLS];`
- A 2D array named identifier is defined as
- a number of ROWS
- Each row is an 1D array of COLS elements of type type
- When initializing during definition:
- It is always required to give the COLS dimension
- The number of ROWS is NOT always required
- it can be calculated based on the number of columns and the initialization.

#234

slide 234

Array of Arrays: Initialization

- `intarr[3][2] = {`
- `{ 1, 4 }`
- `{ 5, 2 }`
- `{ 6, 5 }`
- `};`
- `intarr[3][2] = { 1, 4, 5, 2, 6, 5 };`
- `intarr[3][2] = { { 1, 4 }, { 5, 2 }, { 6, 5 } };`
- Not initialised elements will get 0
- `intarr[3][2] = { { 0 } };`
- All elements are initialised to 0

#235

slide 235

Array of Arrays

- `intarr[4][3];`

- `arr[0][0]`

- `arr[0][2]`

- `arr[0]`

- `arr[1]`

- `arr[3]`

- `arr[1][0]`

- `arr[3][2]`

#236

slide 236

Array of Arrays

- It is continuously stored in memory, row by row

- `intarr[4][3];`

- Memory

- <code>arr[0][0]:</code>	- 0x1000	- 0
- <code>arr[0][1]:</code>	- 0x1004	- 1
- <code>arr[0][2]:</code>	- 0x1008	- 2
- <code>arr[1][0]:</code>	- 0x100C	- 3
		- 4
		- 5
- <code>arr[2][0]:</code>	- 0x1018	- 6
		- 7
		- 8
- <code>arr[3][0]:</code>	- 0x1028	- 9
		- 10
		- 11

#237

slide 237

Array of Arrays: Remarks!

The name of the array is a constant pointer that points to the first element of the array
2 levels of hierarchy
Row
Columns -> elements
first level of hierarchy is the row, so it points to the first row !
arr == &arr[0]
The name of the array is NOT a pointer to pointer to type, (type **)
This is NOT OK with the pointer arithmetic !
The name of the 1D arrays, i.e. the rows, point to the first element of the 1D array
arr[i] == &arr[i][0]

#238

slide 238

Array of Arrays: Pointer arithmetic

- arr[i][j] = *(arr[i] + j)

- intarr[4][3];

- Memory

	- arr[0][0]:	- 0x1000	- 0
	- arr[0][1]:	- 0x1004	- 1
	- arr[0][2]:	- 0x1008	- 2
- arr[i]	- arr[1][0]:	- 0x100C	- 3
			- 4
- arr[i]+j	- arr[1][2]:	- 0x1014	- 5
	- arr[2][0]:	- 0x1018	- 6
			- 7
			- 8
	- arr[3][0]:	- 0x1028	- 9
			- 10
			- 11

- arr[i] points to int

- so, we add: sizeof(int)

#239

slide 239

Anchors: 2D Array: Array of Arrays; Array of Arrays: Initialization; Array of Arrays; Array of Arrays: Remarks!;
Array of Arrays: Pointer arithmetic; Pointer to an array

3.6 Backup: 2D Array: Array of Arrays

Original slide 234

- Syntax:
- type identifier[ROWS][COLS] ;
- A 2D array named identifier is defined as
- a number of ROWS

- Each row is an 1D array of COLS elements of type type
- When initializing during definition:
- It is always required to give the COLS dimension
- The number of ROWS is NOT always required
- it can be calculated based on the number of columns and the initialization.

2D Array: Array of Arrays

- Syntax:
- type identifier[ROWS][COLS];
- A 2D array named identifier is defined as
- a number of ROWS
- Each row is an 1D array of COLS elements of type type
- When initializing during definition:
- It is always required to give the COLS dimension
- The number of ROWS is NOT always required
- it can be calculated based on the number of columns and the initialization.

#234

3.7 Backup: Array of Arrays: Initialization

Original slide 235

- `int arr[3][2] = {`
- `{ 1, 4 },`
- `{ 5, 2 },`
- `{ 6, 5 }`
- `};`
- `int arr[3][2] = { 1, 4, 5, 2, 6, 5 };`
- `int arr[3][2] = {{ 1,4 }, { 5,2 }, { 6,5 }};`
- Not initialised elements will get 0
- `int arr[3][2] = {{ 0 }};`
- All elements are initialised to 0

Array of Arrays: Initialization

```
- intarr[3][2] = {  
- { 1, 4 }  
- { 5, 2 }  
- { 6, 5 }  
- }  
- intarr[3][2] = { 1, 4, 5, 2, 6, 5 };  
- intarr[3][2] = { { 1, 4 }, { 5, 2 }, { 6, 5 } };  
- Not initialised elements will get 0  
- intarr[3][2] = { { 0 } };  
- All elements are initialised to 0
```

#235

3.8 Backup: Array of Arrays

Original slide 236

- `int arr[4][3] ;`
- `arr[0][0]`
- `arr[0][2]`
- `arr[0][1]`
- `arr[0]`
- `arr[1]`
- `arr[2]`
- `arr[3]`
- `arr[1][0]`
- `arr[3][2]`

Array of Arrays

- intarr[4][3];

- arr[0][0]

- arr[0][2]

- arr[0]

- arr[1]

- arr[3]

- arr[1][0]

- arr[3][2]

#236

3.9 Backup: Array of Arrays

Original slide 237

- It is continuously stored in memory, row by row
- `int arr[4][3];`
- Memory
- `arr[0][0]:`
 - 0
 - 0x1000
- `arr`
- `arr[0][1]:`
 - 1
 - 0x1004
- 2
- `arr[0][2]:`
 - 0x1008
- `arr[3]`
- `arr[2]`
- 3
- `arr[1][0]:`
 - `arr[0]`
 - `arr[1]`
 - 0x100C

- 4
- 5
- 6
- arr[2][0]:
- 0x1018
- 7
- 8
- 9
- 0x1028
- arr[3][0]:
- 10
- 11

Array of Arrays

- It is continuously stored in memory, row by row

- intarr[4][3];

- Memory

-arr[0][0]:	-0x1000	-0
-arr[0][1]:	-0x1004	-1
-arr[0][2]:	-0x1008	-2
-arr[1][0]:	-0x100C	-3
		-4
		-5
-arr[2][0]:	-0x1018	-6
		-7
		-8
-arr[3][0]:	-0x1028	-9
		-10
		-11

#237

3.10 Backup: Array of Arrays: Remarks!

Original slide 238

The name of the array is a constant pointer that points to the first element of the array
2 levels of hierarchy

Row

Columns -> elements

first level of hierarchy is the row, so it points to the first row !

arr == &arr[0]

The name of the array is NOT a pointer to pointer to type, (type**)

This is NOT OK with the pointer arithmetic !

The name of the 1D arrays, i.e. the rows, point to the first element of the 1D array

arr[i] == &arr[i][0]

Array of Arrays: Remarks!

The name of the array is a constant pointer that points to the first element of the array
2 levels of hierarchy
Row
Columns -> elements
first level of hierarchy is the row, so it points to the first row !
`arr == &arr[0]`
The name of the array is NOT a pointer to pointer to type, (type **)
This is NOT OK with the pointer arithmetic !
The name of the 1D arrays, i.e. the rows, point to the first element of the 1D array
`arr[i] == &arr[i][0]`

#238

3.11 Backup: Array of Arrays: Pointer arithmetic

Original slide 239

- `arr[i][j] = *(arr[i] + j)`
- `int arr[4][3];`
- Memory
- `arr[0][0]:`
- 0
- 0x1000
- `arr[0][1]:`
- 1
- 0x1004
- 2
- `arr[0][2]:`
- 0x1008
- 3
- `arr[i]`
- `arr[1][0]:`
- 0x100C
- 4
- 5
- `arr[1][2]:`
- 0x1014

- `arr[i]+j`
- 6
- `arr[2][0]:`
- 0x1018
- 7
- `arr[i]` points to int
- 8
- so, we add: `sizeof(int)`
- 9
- 0x1028
- `arr[3][0]:`
- 10
- 11

Array of Arrays: Pointer arithmetic

`- arr[i][j] = *( arr[i] + j)`

`- int arr[4][3];`

`- Memory`

	<code>- arr[0][0]:</code>	<code>- 0x1000</code>	<code>- 0</code>
	<code>- arr[0][1]:</code>	<code>- 0x1004</code>	<code>- 1</code>
	<code>- arr[0][2]:</code>	<code>- 0x1008</code>	<code>- 2</code>
<code>- arr[i]</code>	<code>- arr[1][0]:</code>	<code>- 0x100C</code>	<code>- 3</code>
			<code>- 4</code>
<code>- arr[i]+j</code>	<code>- arr[1][2]:</code>	<code>- 0x1014</code>	<code>- 5</code>
	<code>- arr[2][0]:</code>	<code>- 0x1018</code>	<code>- 6</code>
			<code>- 7</code>
<code>- arr[i] points to int</code>			<code>- 8</code>
<code>- so, we add: sizeof(int)</code>	<code>- arr[3][0]:</code>	<code>- 0x1028</code>	<code>- 9</code>
			<code>- 10</code>
			<code>- 11</code>

#239

3.12 Backup: Array of Arrays: Pointer arithmetic

Original slide 240

- `arr[i][j] = *( arr[i] + j)`
- `*( *(arr + i) + j)`
- `int arr[4][3];`
- Memory
- `arr[0][0]:`
- 0
- arr

- 0x1000
- arr points to 1D array of int
- arr[0][1]:
- 1
- 0x1004
- !!!
- 2
- arr[0][2]:
- 0x1008
- 3
- arr+i
- arr[1][0]:
- 0x100C
- so, we add the size of the array: sizeof(int)*COLS
- 4
- 5
- arr[1][2]:
- 0x1014
- *(arr+i)+j
- 6
- arr[2][0]:
- 0x1018
- 7
- *(arr+i) is arr[i] i.e. 1D array of int
- 8
- so, we add: sizeof(int)
- 9
- 0x1028
- arr[3][0]:
- 10
- 11

Array of Arrays: Pointer arithmetic

- arr[i][j] = *(arr[i] + j)				
- *(*(arr + i) + j)				
				- intarr[4][3];
				- Memory
- arr points to 1D array of int	- arr	- arr[0][0]:	- 0x1000	- 0
- !!!		- arr[0][1]:	- 0x1004	- 1
		- arr[0][2]:	- 0x1008	- 2
	- arr+i	- arr[1][0]:	- 0x100C	- 3
so, we add the size of the array:				- 4
sizeof(int)*COLS		- arr[1][2]:	- 0x1014	- 5
	- *(arr+i)+j	- arr[2][0]:	- 0x1018	- 6
				- 7
- *(arr+i) is arr[i] i.e. 1D array				- 8
of int				- 9
- so, we add: sizeof(int)		- arr[3][0]:	- 0x1028	- 10
				- 11

#240

3.13 Backup: Array of Arrays: Pointer arithmetic

Original slide 241

- $\text{arr}[i][j] = *(\text{arr}[i] + j)$
- $*(*(\text{arr} + i) + j)$
- $*((* \text{arr}) + i * \text{COLS} + j)$
- `int arr[4][3];`
- Memory
- `arr[0][0]:`
- 0
- `arr`
- 0x1000
- `arr` points to 1D array of int
- `arr[0][1]:`
- 1
- 0x1004
- 2
- `arr[0][2]:`
- 0x1008
- 3
- $* \text{arr} + i * \text{COLS}$
- `arr[1][0]:`
- 0x100C

- 4
- Manually add the size of the array instead of compiler
- 5
- `arr[1][2]:`
- `0x1014`
- `(*arr)+i*COLS+j`
- 6
- `arr[2][0]:`
- `0x1018`
- 7
- 8
- 9
- `0x1028`
- `arr[3][0]:`
- 10
- 11

Array of Arrays: Pointer arithmetic				
<code>- arr[i][j] = *(arr[i] + j)</code> <code>- *(*(arr + i) + j)</code>				
				- <code>intarr[4][3];</code>
				- Memory
	- <code>arr</code>	- <code>arr[0][0]:</code>	- <code>0x1000</code>	- 0
- <code>arr</code> points to 1D array of int		- <code>arr[0][1]:</code>	- <code>0x1004</code>	- 1
		- <code>arr[0][2]:</code>	- <code>0x1008</code>	- 2
	- <code>*arr+i*COLS</code>	- <code>arr[1][0]:</code>	- <code>0x100C</code>	- 3
- Manually add the size of the array instead of compiler		- <code>arr[1][2]:</code>	- <code>0x1014</code>	- 4
	- <code>(*arr)+i*COLS+j</code>	- <code>arr[2][0]:</code>	- <code>0x1018</code>	- 5
				- 6
				- 7
				- 8
		- <code>arr[3][0]:</code>	- <code>0x1028</code>	- 9
				- 10
				- 11

#241

3.14 Backup: Pointer to an array

Original slide 242

- `int arr[ROWS][COLS];`
- `int (*ptr) [COLS] ;`
- `ptr = arr;`
- `(*ptr)` is a pointer to an 1D array of COLS elements. The parentheses are NOT optional !
- Without the parentheses, `ptr` becomes an array of COLS pointers, not a pointer to an array of COLS ints
- Doing pointer arithmetic, we can compute correctly the address of `ptr[row][col]` :

- $\text{ptr}[\text{row}][\text{col}] = \text{ptr} + \text{sizeof}(\text{int}) * \text{COLS} * \text{row}$
- ▶ $\text{sizeof}(\text{int}) * \text{col}$

Pointer to an array

```
- int arr[ROWS][COLS];
- int (*ptr) [COLS];
- ptr = arr;
- (*ptr) is a pointer to an 1D array of COLS elements. The parentheses are NOT optional !
- Without the parentheses, ptr becomes an array of COLS pointers, not a pointer to an array of COLS ints
- Doing pointer arithmetic, we can compute correctly the address of ptr[row][col] :
- ptr[row][col] = ptr + sizeof(int) * COLS * row
- + sizeof(int) * col
```

#242

3.15 Backup: Let's...

Original slide 243

- Kahoot time!
- Q12 - Q19

Let's...

```
- Kahoot time!
- Q12 - Q19
```

#243